

EE108 Final Project Checkpoint 0

Berwyn Berwyn
Stanford University

Arif Ismail
Stanford University

1 Chords

1.1 What it does

This is a required extension. The music player needs to be capable of playing multiple (at least 3) notes simultaneously.

1.2 Expected points

We expect 4 points, following the final project specification.

1.3 Implementation idea

1. Our design will need to generate multiple waveforms (one for each note in the chord) and combine them into a single audio sample to feed to the codec.

2 Enhanced Waveform Display

2.1 What it does

Each note in the generated chord will be displayed in a different color, with the combined waveform in white.

2.2 Expected points

We expect 1 point, following the final project specification.

2.3 Implementation idea

1. We can hard-code the color assigned to a note and implement a "Color ROM" that takes in a note and output the color associated with it.

3 Rewinding

3.1 What it does

At any moment in time, we can pause the player and press a rewind button to play the song in reverse.

3.2 Expected points

We expect 4 points, following the final project specification.

3.3 Implementation idea

1. In the song reader module, we will have 2 modes: **normal** and **rewind**. When the mode is normal, we increment the note counter by one and when rewind, we decrement the note counter by one. This way, we can traverse the song ROM in both directions.
2. In sine reader, we also need to sample the wave in reverse order, this is straightforward as we only need to mirror our sampling process with respect to the x axis.

4 Harmonics

4.1 What it does

This function allows adding higher frequency harmonics to simulate different instrument sounds. If time permits, we might provide a user interface for users to choose the type of harmonics to play.

4.2 Expected points

We expect 3 points, following the final project specification.

4.3 Implementation idea

1. For a note with frequency f , we sample higher frequencies nf where n is an integer (specifics regarding what frequencies to include are to be researched).
2. We sample each frequency components from the sine ROM.
3. We finally combine all the samples via a weighted sum, which will give us a sound with multiple frequency components.

5 Neural Network for Melody Generator

5.1 What it does

Given the first n notes and durations, we use a tiny feed-forward neural network to predict the next sequence of notes and durations in real time and play the song.

5.2 Expected points

We expect 5 points for this extension, as we need to build the feed forward neural network and add real-time interactivity with the users in order to generate the song

5.3 Implementation idea

1. We can initially train a neural network model in Python to predict the next note given the previous k notes (the value of k is for us to engineer later).
2. We can get the weights of the model and hard code them into the FPGA. This allows the FPGA to grab the k notes in the song ROM and predict the next note.